

PathCondRAG: Path-Conditioned Query Decomposition for Multi-Hop Retrieval-Augmented Generation

Anonymous Author(s)

Abstract

Multi-hop retrieval often fails not because the first relevant passage is unreachable, but because the next passage depends on a bridge entity that is only revealed after retrieval has begun. Query decomposition can rewrite a complex question into smaller sub-questions, yet a decomposer that only sees the original question may keep references such as “the director” or “that country” unresolved. Graph-enhanced RAG provides a useful but underused signal here: an initial graph pass can expose candidate bridge entities, supporting snippets, and evidence paths before the next query is written. PathCondRAG uses this signal to condition query decomposition. It extracts compact path hints, each containing a source entity, a bridge entity, a snippet, and a confidence score, and supplies them to an LLM decomposer so that generated sub-questions name corpus-grounded entities. Formally, the intervention changes decomposition from $f(q)$ to $f(q, \mathcal{P})$. We evaluate PathCondRAG on Natural Questions, PopQA, MuSiQue, HotpotQA, and 2WikiMultiHopQA against standard retrievers, iterative retrieval, and structure-aware RAG baselines. The gains concentrate on multi-hop benchmarks, where path conditioning improves early evidence recall and downstream QA. NQ and PopQA show a more limited transfer setting: the method can improve recall without always improving exact answer generation. Controlled ablations further show that graph-derived path hints contribute more than enabling decomposition alone. The anonymized code repository is available at <https://anonymous.4open.science/r/PathCondRAG>.

CCS Concepts

• Information systems → Information retrieval.

Keywords

Retrieval-Augmented Generation, Multi-hop Question Answering, Query Decomposition, Graph-enhanced Retrieval, Path-conditioned Retrieval

ACM Reference Format:

Anonymous Author(s). 2018. PathCondRAG: Path-Conditioned Query Decomposition for Multi-Hop Retrieval-Augmented Generation. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Retrieval-augmented generation (RAG) addresses a practical limitation of closed-book language models: factual associations stored in parameters are incomplete, popularity-biased, and difficult to update [25, 28, 31]. Retrieval-augmented models attach a non-parametric memory to the generator, from open-domain systems such as REALM, RAG, and FiD to large-scale retrieval-augmented language models such as RETRO and Atlas [3, 10, 12, 13, 22].

Multi-hop retrieval is a stricter version of this problem because the evidence needed to answer a question is often distributed across several documents. A system must retrieve passages that are individually relevant and assemble them into a chain of complementary evidence. Benchmarks such as MuSiQue, HotpotQA, and 2WikiMultiHopQA were designed to test this chain property under different multi-hop reasoning patterns [11, 39, 44]. In this setting, retrieval and reasoning are tightly coupled: later evidence often becomes retrievable only after an entity or relation has been discovered in earlier evidence.

Standard sparse, dense, and late-interaction retrievers rank passages by relevance to the original query [17, 18, 32, 33]. This objective works well when the required information is expressed in a single passage or shares clear lexical or semantic overlap with the question. In multi-hop questions, however, a later evidence passage may become retrievable only after a bridge entity has been identified. For example, answering “What is the capital of the country where the director of *Spirited Away* was born?” requires first resolving the director, then using that bridge entity to retrieve the birthplace and capital facts. Without this resolved bridge entity, the second-hop query remains underspecified.

Query decomposition and iterative retrieval address part of this problem by rewriting the question before or during retrieval. Prior work shows that intermediate questions or generated search queries can help recover missing entities in multi-hop QA [26, 30, 43], while recent RAG methods use query rewriting, hypothetical documents, or adaptive retrieval to better align retrieval with the reader’s information need [2, 6, 14, 15, 24]. However, the feedback signal in these methods is usually the original question, generated text, or free-form reasoning context. It is not the structured set of bridge entities and evidence paths exposed by graph retrieval. This weak coupling can leave sub-questions underspecified, causing references such as “the director” or “that country” to remain unresolved or to drift toward plausible but unsupported entities.

Graph-enhanced RAG offers a complementary signal. Entity–passage graphs, passage graphs, communities, and path-based retrieval methods can expose bridge entities and candidate evidence paths during retrieval [4, 5, 8, 9]. These structures make intermediate entities explicit rather than leaving them implicit in retrieved text. However, the graph signal is usually used to rank passages,

traverse graph context, or support generation. When query decomposition is used, the decomposer often remains weakly coupled with the path-level structure revealed by graph retrieval.

We target this interface between retrieval and decomposition. PathCondRAG first runs graph retrieval to expose candidate bridge entities and supporting paths. It then compresses these structures into path hints, each consisting of a source entity, a bridge entity, a supporting snippet, and a confidence score. These hints are inserted before query decomposition rather than used only as final evidence context. The decomposer receives the original question together with the hints, so its sub-questions can name corpus-grounded entities rather than unresolved roles. The resulting sub-questions are used for a second evidence retrieval pass and fused with the initial graph-retrieval results before answer generation. At the decomposition step, the intervention changes the input from the question alone, $f(q)$, to the question plus graph-derived path hints, $f(q, \mathcal{P})$.

We evaluate PathCondRAG primarily on MuSiQue, HotpotQA, and 2WikiMultiHopQA, three widely used multi-hop benchmarks that directly test bridge-dependent evidence retrieval [11, 39, 44]. PathCondRAG improves early evidence recall and downstream QA over graph-enhanced retrieval and static decomposition variants on these datasets. We also include Natural Questions and PopQA as factual and long-tail transfer settings [20, 25]. Their results show a more limited transfer pattern: path conditioning can improve recall, but recall gains do not always translate into better exact answer generation. Controlled ablations further show that the main gain comes from conditioning decomposition on graph-derived path hints rather than from enabling decomposition alone.

Our contributions are:

- We formulate path-conditioned query decomposition for multi-hop retrieval, shifting decomposition from $f(q)$ to $f(q, \mathcal{P})$, where $\mathcal{P}$ contains graph-derived path hints.
- We implement PathCondRAG, a retrieval-decomposition-retrieval pipeline that uses source entities, bridge entities, snippets, and confidence scores to generate grounded sub-questions.
- We show that the gains concentrate on bridge-dependent multi-hop settings and are mainly explained by path-conditioned decomposition, supported by controlled ablations, hop-stratified analysis, path-hint diagnostics, and failure analysis.

2 Related Work

PathCondRAG sits between two lines of work that are usually studied separately: query-side decomposition and structure-aware retrieval. We review multi-hop QA and RAG retrieval first, then discuss decomposition and iterative retrieval, and finally compare with graph-enhanced and path-based RAG systems.

2.1 Multi-hop RAG and Complex Question Answering

Multi-hop question answering requires models to combine evidence from multiple passages rather than answer from a single retrieved context. Early resources such as WikiHop highlighted the need to combine disjoint evidence across documents [42], while TriviaQA

and Natural Questions made open-domain evidence retrieval central to QA evaluation [16, 20]. Datasets such as MuSiQue, HotpotQA, and 2WikiMultiHopQA were later designed to test multi-hop composition under different reasoning patterns [11, 39, 44]. In RAG systems, this setting is challenging because retrieval errors compound: if an early bridge entity is missed, later evidence may never enter the reader context.

Evaluation suites such as KILT and BEIR broaden this picture by testing whether retrieval systems generalize across knowledge-intensive tasks and out-of-distribution retrieval settings [27, 38]. PopQA adds another stressor: long-tail factual questions where parametric memory is unreliable and retrieval becomes especially important [25]. PathCondRAG uses these datasets differently from the multi-hop benchmarks. The core mechanism is designed for bridge recovery in multi-hop QA, while NQ and PopQA test whether the added decomposition step preserves factual retrieval behavior outside the primary setting.

Sparse, dense, and late-interaction retrievers provide strong passage-level matching, but usually score passages against the current query rather than optimize evidence-chain completeness [17, 18, 32, 33]. Retrieval-augmented readers further show that retrieved context improves open-domain QA and knowledge-intensive generation [12, 13, 22]. PathCondRAG builds on this retrieval-reader setting, but focuses on a different failure mode: later evidence may require a bridge entity that is only exposed after an initial retrieval pass.

2.2 Query Decomposition and Iterative Retrieval

Query decomposition and iterative retrieval methods reduce a complex question into intermediate steps. DecompRC decomposes multi-hop reading comprehension questions into simpler sub-questions and globally rescores decompositions [26]. GoldEn Retriever iteratively generates search queries to recover missing entities in open-domain multi-hop QA [30], and BREAK provides a benchmark for evaluating question decomposition quality [43]. Recent prompting work such as Decomposed Prompting and least-to-most prompting also shows the value of solving complex tasks through simpler sub-problems [19, 47]. Broader reasoning-time prompting and tool-use methods also use intermediate steps or external actions to support complex problem solving [35, 41, 45]. More retrieval-oriented methods are closer to our setting: Self-Ask encourages a model to ask follow-up questions, while IRCoT interleaves chain-of-thought reasoning with retrieval [29, 40]. These methods create a feedback loop between partial reasoning and later retrieval, but the feedback is usually expressed as generated text or free-form reasoning context rather than structured graph-derived path hints.

The feedback signal in these methods is usually expressed as generated text, intermediate reasoning context, or a rewritten query. Query rewriting, HyDE-style hypothetical documents, FLARE, Self-RAG, and Adaptive-RAG all demonstrate that changing when and how retrieval is invoked can improve knowledge access [2, 6, 14, 15, 24]. This can be effective, but it does not guarantee that bridge entities discovered by retrieval are represented as explicit conditioning variables for decomposition. PathCondRAG targets this interface: once graph retrieval reveals candidate bridge entities

and supporting snippets, the decomposer should receive them as structured conditioning variables rather than infer them from the original question or free-form reasoning text.

2.3 Graph-enhanced Retrieval and Graph-based RAG

Graph-enhanced RAG methods use structure to connect entities, passages, or communities. Earlier QA systems such as GRAFT-Net, PullNet, and graph-based reasoning-path retrieval already showed that question-specific graphs can combine textual and structured evidence [1, 36, 37]. QA-GNN similarly connects language context with knowledge-graph nodes for joint reasoning [46]. HippoRAG builds an entity-passage graph and retrieves through graph propagation [8], while HippoRAG 2 further develops memory-style retrieval [9]. GraphRAG uses graph communities and summaries for query-focused generation [5]. RAPTOR builds hierarchical document representations, making it structure-aware but not an entity-graph method [34].

These methods show that structure can improve retrieval and generation. The question studied here is different: once graph retrieval exposes intermediate structure, how should that structure affect query decomposition? In contrast, PathCondRAG does not use graph structure only to expand, rank, or summarize retrieval context. It uses graph-exposed paths before the decomposer writes the next query, so the graph signal changes the query-generation step itself.

2.4 Path-based and Entity-centric Evidence Modeling

Path-based RAG methods are the closest neighbors of this work. PathRAG and HopRAG use paths or traversal policies to select graph context for retrieval and generation [4, 23]. LightRAG and CatRAG similarly emphasize efficient or context-aware graph traversal over entity- or relation-centric structures [7, 21]. These methods mainly decide which graph context, paths, or entity-linked evidence should be retrieved, pruned, traversed, or passed to the generator.

PathCondRAG uses path information at a different point in the pipeline. It does not treat paths only as evidence context, traversal traces, or pruning targets. Instead, it compresses retrieved paths into compact hints and supplies them to the query decomposer before the next retrieval query is generated. The path signal therefore changes the decomposition step itself: the conditioned sub-questions determine the next retrieval trajectory rather than merely consuming graph paths after retrieval.

3 Preliminaries

We first define the retrieval setting, the entity–passage graph, and the graph propagation step used to obtain candidate bridge evidence.

3.1 Problem Setup

Let q be a multi-hop question and let $C = \{d_1, \dots, d_N\}$ denote a corpus of passages. The retriever returns a ranked list $\mathcal{D}_k = (d_{(1)}, \dots, d_{(k)})$, and a reader consumes $\mathcal{D}_k$ to produce an answer. We focus on settings where the evidence needed to answer q is

distributed across passages, so a useful retrieval result must cover an evidence chain rather than only the passage most similar to q .

We distinguish two decomposition regimes. In unconditioned decomposition, a decomposer generates sub-questions from the question alone, written as $f(q)$. In path-conditioned decomposition, the decomposer additionally receives graph-derived path hints $\mathcal{P}$ and generates $f(q, \mathcal{P})$. The method section defines how $\mathcal{P}$ is constructed and used for retrieval.

3.2 Entity–Passage Graph

We represent the corpus as a bipartite graph $G = (V_E \cup V_D, E)$, where V_E is the set of entities extracted from the corpus and V_D is the set of passages. An undirected edge connects an entity $e \in V_E$ to a passage $d \in V_D$ whenever e is mentioned in or extracted from d .

This graph structure provides a substrate for cross-document connections through shared entities. In particular, intermediate entities reached by graph propagation can serve as candidate bridge entities linking different pieces of evidence. These bridge entities are important for multi-hop retrieval because they often correspond to the implicit references that unconditioned decomposers struggle to resolve.

3.3 Personalized PageRank

To obtain initial passages and candidate bridge entities, PathCondRAG performs retrieval over the entity–passage graph using Personalized PageRank (PPR). Let $\mathbf{s}$ be a personalization vector initialized on query seed entities extracted from q , and let $\mathbf{M}$ be the row-normalized transition matrix of G . The PPR iteration is

$$\mathbf{r}^{(t+1)} = \alpha \mathbf{s} + (1 - \alpha) \mathbf{M}^\top \mathbf{r}^{(t)}, \quad (1)$$

where $\alpha \in (0, 1]$ controls the restart probability.

Unlike passage-level matching against q alone, PPR propagates relevance through the graph topology. High-scoring nodes can therefore include directly relevant passages, intermediate entities, and graph connections that suggest candidate evidence paths. PathCondRAG uses these nodes and connections to construct path hints $\mathcal{P}$, which are later provided to the decomposer. The next section defines the hint representation and selection procedure.

4 Method

PathCondRAG is a retrieval-decomposition-retrieval pipeline. The section below follows the order in which a query is processed: initial graph retrieval, path hint extraction, path-conditioned decomposition, evidence retrieval, score fusion, and answer generation.

4.1 Overview

PathCondRAG takes a multi-hop question q and returns ranked evidence passages for answer generation. Its key design is to insert a structured feedback step between initial graph retrieval and query decomposition. Rather than decomposing q in isolation, the system first retrieves over an entity–passage graph, extracts compact path hints, and then generates sub-questions conditioned on those hints.

The pipeline has four stages: initial graph retrieval, path hint extraction, path-conditioned query decomposition, and evidence

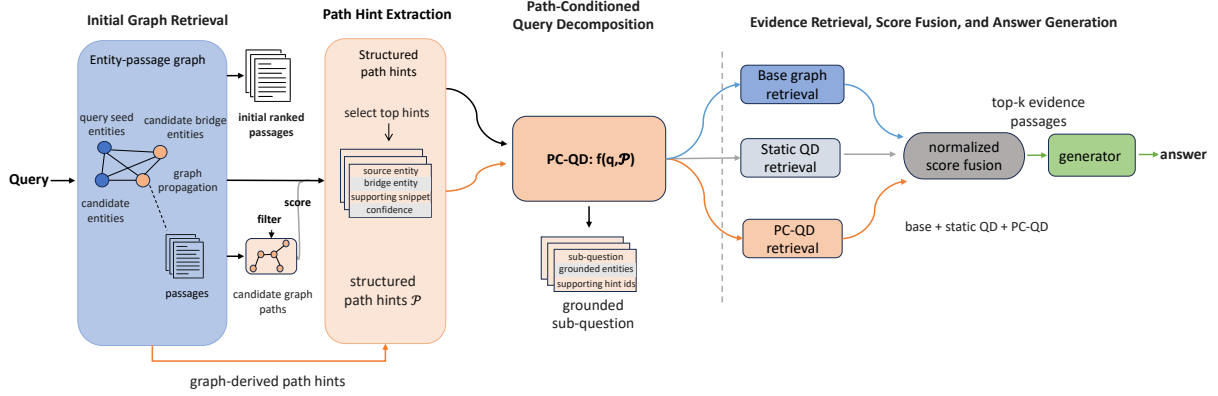


Figure 1: Overview of PathCondRAG. Given a multi-hop query, PathCondRAG first performs initial graph retrieval over an entity–passage graph to expose candidate bridge entities, ranked passages, and graph paths. It then extracts compact structured path hints and uses them to condition query decomposition, changing the decomposer from an unconditioned mapping $f(q)$ to a path-conditioned mapping $f(q, \mathcal{P})$. The generated grounded sub-questions are used for evidence retrieval, and their scores are fused with base graph retrieval and static decomposition retrieval before answer generation.

retrieval with score fusion. The graph signal is used before decomposition rather than only after retrieval as reader context or reranking evidence. This placement distinguishes PathCondRAG from a simple reranking pipeline: path hints change the queries used for the next retrieval pass. Figure 1 summarizes the full pipeline.

4.2 Initial Graph Retrieval and Path Hint Extraction

The initial graph retrieval pass surfaces candidate bridge entities before query decomposition. Given a query q , the system extracts query seed entities E_q and runs graph retrieval over the entity–passage graph G . The top retrieved passages and high-scoring intermediate entities form a candidate path set.

We consider two lightweight path types. A direct path connects a query seed entity to a passage containing it, (e_i^{src}, d) . A bridge path connects a seed entity to a candidate bridge entity and then to a passage containing that bridge, $(e_i^{\text{src}}, e_i^{\text{br}}, d)$. Each candidate path is associated with the passage it supports and the query seed entities it covers.

Each path hint is represented as

$$h_i = (e_i^{\text{src}}, e_i^{\text{br}}, x_i, c_i), \quad (2)$$

where e_i^{src} is a query seed entity, e_i^{br} is a candidate bridge entity, x_i is a short supporting snippet from the associated passage, and c_i is the hint confidence. For direct paths, where no explicit bridge is found, the bridge field is left empty and the hint acts as a direct grounding signal.

We score each candidate using four normalized components: passage relevance, path connectivity, seed–bridge consistency, and seed–entity coverage. After min–max normalization within the

candidate set, the confidence score is

$$c_i = \lambda_{\text{rel}} r_i + \lambda_{\text{conn}} g_i + \lambda_{\text{cons}} s_i + \lambda_{\text{comp}} u_i, \quad (3)$$

where r_i is the supporting passage’s retrieval score, g_i reflects the candidate’s path connectivity, s_i measures consistency between the query seed distribution and the bridge signal, and u_i measures coverage of query seed entities. The weights are fixed across datasets.

To keep the conditioning signal compact, we greedily select a small set of hints. At each step, we choose the candidate that maximizes

$$\Delta(h_i) = (1 - \beta) c_i + \beta \frac{|(E_i \setminus E_{\text{covered}}) \cap E_q|}{|E_q|}, \quad (4)$$

where E_i is the entity set covered by candidate h_i , E_{covered} is the entity set already covered by selected hints, and β controls the confidence–coverage trade-off. Candidates whose supporting passage has already been selected are skipped. This produces a compact, non-duplicate set of path hints.

Figure 3 illustrates this construction process. Query seed entities initialize local graph retrieval; selected candidate paths are compressed into source–bridge–snippet–confidence hints. When no strict bridge path is available, the system falls back to document-level co-occurrence or relaxed entity overlap under the same selection policy. These cases are recorded in the retrieval diagnostics.

4.3 Path-conditioned Query Decomposition

Path-conditioned query decomposition (PC-QD) is the core mechanism of PathCondRAG. Figure 2 gives a mechanism-level comparison between static query decomposition and PC-QD. The left side shows the static setting, where the decomposer receives only the original question q and generates $f(q)$. Because no retrieved bridge

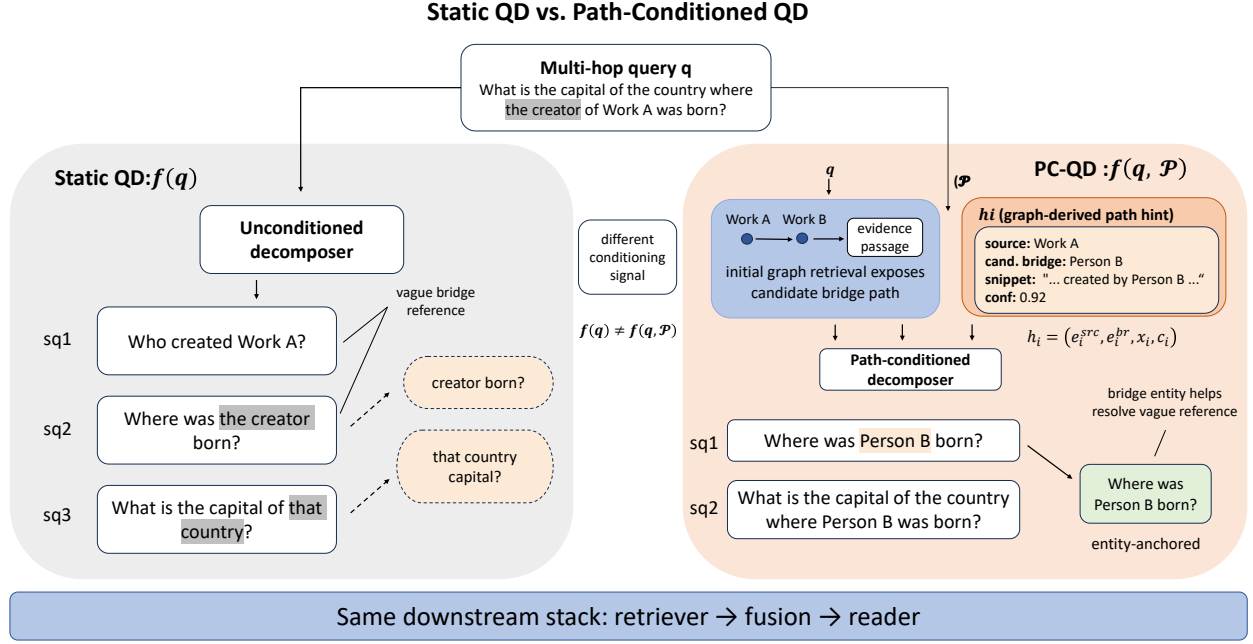


Figure 2: Mechanism of path-conditioned query decomposition. Static QD maps only the original question to sub-questions, $f(q)$, leaving bridge references underspecified. PC-QD conditions the same decomposer on graph-derived path hints, $f(q, \mathcal{P})$, grounding sub-questions to explicit bridge entities before re-retrieval. The downstream retriever, fusion, and reader are held fixed; the isolated change is the conditioning signal used by decomposition.

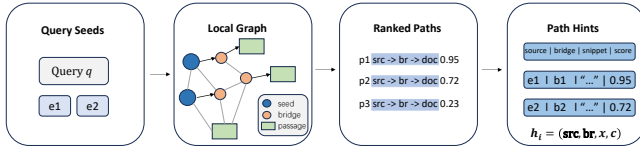


Figure 3: Path hint construction. Query seed entities initialize local graph retrieval; selected paths are compressed into source–bridge–snippet–confidence hints. The local graph is shown as a compressed visualization of entity–passage connectivity, where entity-to-entity links denote paths mediated by passages.

entity is supplied, the generated sub-questions may preserve vague references such as “the creator” or “that country.” These unresolved references then become low-specificity retrieval queries, making the next-hop evidence harder to retrieve.

The right side of Figure 2 shows the path-conditioned setting. Before decomposition, initial graph retrieval exposes candidate bridge paths and compresses them into structured path hints. PC-QD receives both the original question and the hint set $\mathcal{P}$, producing $f(q, \mathcal{P})$. Each hint provides a source entity, a candidate bridge entity, a supporting snippet, and a confidence score. These hints do not answer the question directly; instead, they provide explicit bridge

entities that help the decomposer replace vague references with entity-grounded sub-questions.

The decomposer is prompted to produce at most three structured records:

(sub-question, grounded entities, supporting hint ids, confidence).

The main constraint is that sub-questions should refer to concrete entities from the hints when those entities are relevant. For example, instead of generating an underspecified sub-question such as “Where was the creator born?”, PC-QD can generate an entity-grounded sub-question such as “Where was Person B born?” when a path hint links the source entity to Person B. As shown at the bottom of Figure 2, the downstream retriever, fusion module, and reader are held fixed; the isolated change is the conditioning signal used by decomposition.

PC-QD does not force the decomposer to use every hint. The confidence score and supporting snippet help order the hints, while the original question remains available to preserve the intended semantics. Invalid or empty outputs are discarded. If PC-QD produces no usable sub-question or no path hint is available, the system falls back to the static decomposition branch.

4.4 Evidence Retrieval and Answer Generation

PathCondRAG retrieves evidence from three query sources: the original question, static sub-questions, and path-conditioned sub-questions. For each source, retrieval produces a score vector over the

Table 1: Dataset summary for the main evaluation. NQ and PopQA test factual and long-tail transfer, while MuSiQue, HotpotQA, and 2WikiMultiHopQA are the primary multi-hop benchmarks.

Dataset	Task type	Questions	Corpus	Main stressor	Role
NQ	Open-domain QA	1,000	9,633	factual passage retrieval	transfer
PopQA	Long-tail entity QA	1,000	8,676	entity popularity and tail facts	transfer
MuSiQue	Compositional multi-hop QA	1,000	11,656	composed 2–4 hop evidence chains	primary
HotpotQA	Bridge/comparison multi-hop QA	1,000	9,811	bridge and comparison reasoning	primary
2WikiMultiHopQA	Relation-chain multi-hop QA	1,000	6,119	explicit multi-hop relation paths	primary

candidate pool. We min–max normalize each branch score vector before fusion and merge them as

$$\text{score}(d) = w_{\text{base}}s_{\text{base}}(d) + w_{\text{static}}s_{\text{static}}(d) + w_{\text{path}}s_{\text{pcqd}}(d). \quad (5)$$

Here s_{base} , s_{static} , and s_{pcqd} denote the normalized scores from initial graph retrieval, static decomposition retrieval, and path-conditioned decomposition retrieval, respectively. We use fixed branch weights $(w_{\text{base}}, w_{\text{static}}, w_{\text{path}}) = (0.4, 0.2, 0.4)$ for all datasets and keep them unchanged in the main experiments and ablations. The static branch is retained because path hints can be missing or noisy, and it provides a controlled fallback to unconditioned decomposition.

The reader receives the top-5 passages from the merged ranking. During retrieval, PathCondRAG first collects a top-200 candidate pool and allows the decomposer to generate at most three sub-questions, with up to three retrieved passages per sub-question. These settings are fixed across all datasets.

This placement is important for isolating the contribution of PC-QD. If hints were appended directly to the reader context, gains could come from exposing answer-bearing spans to the generator. If all retrieved passages were sent to the decomposer, gains could come from a larger context window rather than from path structure. PathCondRAG instead provides a compact structured object to the decomposer and keeps the retriever-reader stack fixed. The comparison with static QD therefore tests whether graph-derived path hints change the retrieval trajectory before generation.

This query-side placement also explains why the method helps unevenly across datasets. On single-hop factual questions, a strong direct retriever may already issue a sufficiently specific query, so additional decomposition can add latency without changing the answer-bearing evidence. On multi-hop questions, however, the original query often contains an unresolved role, event, or relation. In those cases, a path hint can convert a vague second-hop query into an entity-grounded one, increasing the chance that the missing bridge evidence appears within the reader’s top-5 context.

4.5 Optional Multi-path Consensus Reranking

Multi-path consensus evidence (MPCE) is an optional reranking step. It checks whether a candidate passage is supported by multiple high-IDF entities or paths and applies a capped score adjustment to already retrieved candidates. MPCE does not introduce new passages. Its role is to test whether agreement among multiple paths can slightly improve the ordering of second-hop evidence.

Because the observed gains from MPCE are small and sometimes mixed across metrics, we treat it as an auxiliary analysis. The core method and contribution remain path-conditioned query decomposition.

5 Experiments

The empirical evaluation asks whether conditioning query decomposition on graph-derived path hints improves multi-hop retrieval and downstream QA beyond static decomposition and graph retrieval alone. We first compare against standard, iterative, and structure-aware baselines on five datasets using completed runs only. We then isolate PC-QD with controlled ablations, analyze where the gains concentrate, and close with failure and efficiency evidence.

5.1 Experimental Setup

We evaluate on five QA datasets: Natural Questions (NQ) [20], PopQA [25], MuSiQue [39], HotpotQA [44], and 2WikiMultiHopQA [11]. Each processed evaluation file contains 1,000 questions, and all methods are evaluated on the same files with full-corpus indexing. The retriever returns a top-200 candidate pool, and the reader receives the top-5 passages from the final ranking. We report Recall@5 and Recall@10 for retrieval, and Exact Match (EM) and F1 for downstream QA. All iterative or decomposition-based methods use a maximum step budget of 3.

The reader and decomposition model use Qwen3-30B-A3B, and Qwen-Embedding denotes Qwen3-Embedding-0.6B.¹ The main comparison includes three baseline families. Standard retrieval baselines are BM25, Contriever, and Qwen-Embedding. Decomposition and iterative baselines are Static QD with Qwen-Embedding, Static QD with HippoRAG2, and IRCot. Structure-aware baselines are RAPTOR and HippoRAG2.

All reported methods use the same 1,000-question evaluation files, reader, top-5 evidence budget, and evaluation script. When a baseline requires its own indexing pipeline, we align the retrieval and reader budgets, and treat the result as a reference comparison rather than a strict component-level ablation.

Table 1 defines the evaluation scope. MuSiQue, HotpotQA, and 2WikiMultiHopQA are the primary benchmarks because they require multi-hop evidence assembly and directly test whether path-conditioned decomposition improves bridge recovery. NQ and PopQA are included as transfer checks: they test whether the same pipeline remains competitive when the question is factual or long-tail rather than explicitly compositional.

5.2 Main Results

Table 2 shows that PathCondRAG improves early evidence recall most clearly on the primary multi-hop datasets. On MuSiQue, it raises R@5 from the strongest non-ours score of 0.5858 to 0.6114

¹Qwen3-30B-A3B refers to Qwen3-30B-A3B-Instruct-2507-FP8; Qwen-Embedding refers to Qwen3-Embedding-0.6B.

Table 2: Main retrieval results on five QA datasets. All values use the same 1,000-question evaluation files and exact-match evidence recall. Best values are bolded.

Method	NQ		PopQA		MuSiQue		2Wiki		HotpotQA	
	R@5	R@10	R@5	R@10	R@5	R@10	R@5	R@10	R@5	R@10
<i>Standard Retrieval</i>										
BM25	0.5505	0.8070	0.3935	0.4910	0.4326	0.5107	0.6583	0.7065	0.7290	0.8595
Contriever	0.6188	0.8922	0.4860	0.5035	0.5289	0.6217	0.6667	0.7220	0.8050	0.8760
Qwen-Emb.	0.6372	0.9042	0.5005	0.5075	0.5578	0.6350	0.6987	0.7298	0.8040	0.8740
<i>Decomposition / Iterative Retrieval</i>										
Static QD + Qwen-Emb.	0.6509	0.9108	0.4990	0.5295	0.5858	0.6737	0.7160	0.7610	0.8340	0.9065
Static QD + HippoRAG2	0.6467	0.9106	0.4995	0.5295	0.5836	0.6741	0.7175	0.7612	0.8380	0.9045
IRCoT	0.5479	0.8061	0.3660	0.4835	0.4517	0.6595	0.6573	0.8928	0.7335	0.9205
<i>Structure-Aware Retrieval</i>										
RAPTOR	0.6226	0.8703	0.4835	0.5000	0.5085	0.5832	0.6625	0.7030	0.7495	0.8260
HippoRAG2	0.6507	0.9058	0.4975	0.5195	0.5780	0.6562	0.7405	0.7708	0.8415	0.8990
PathCondRAG	0.6402	0.9031	0.5135	0.5625	0.6114	0.7178	0.8107	0.8762	0.8415	0.9260

Table 3: Downstream QA results with the reader restricted to the top-5 retrieved passages. All methods use the same 1,000-question evaluation files, reader, and evidence budget. Best values are bolded.

Method	NQ		PopQA		MuSiQue		2Wiki		HotpotQA	
	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
<i>Standard Retrieval</i>										
BM25	0.3870	0.5231	0.4320	0.5580	0.1790	0.2795	0.4590	0.5336	0.4830	0.6098
Contriever	0.3900	0.5294	0.4490	0.5889	0.2480	0.3502	0.4420	0.5218	0.4880	0.6302
Qwen-Emb.	0.4120	0.5397	0.4400	0.5822	0.2350	0.3412	0.4930	0.5723	0.4990	0.6335
<i>Decomposition / Iterative Retrieval</i>										
Static QD + Qwen-Emb.	0.3980	0.5312	0.4110	0.5254	0.2480	0.3512	0.5120	0.5924	0.5190	0.6527
Static QD + HippoRAG2	0.4010	0.5316	0.4090	0.5235	0.2530	0.3598	0.5120	0.5901	0.5220	0.6517
IRCoT	0.3070	0.4330	0.4560	0.5500	0.2580	0.3504	0.4020	0.5270	0.4560	0.5847
<i>Structure-Aware Retrieval</i>										
RAPTOR	0.3850	0.5237	0.4420	0.5776	0.2180	0.3170	0.4630	0.5408	0.4770	0.6080
HippoRAG2	0.3920	0.5297	0.4100	0.5217	0.2260	0.3273	0.5110	0.5969	0.5140	0.6480
PathCondRAG	0.3900	0.5240	0.4100	0.5225	0.2700	0.3714	0.5440	0.6356	0.5190	0.6536

and R@10 from 0.6741 to 0.7178. On 2WikiMultiHopQA, it improves R@5 from 0.7405 to 0.8107, although IRCoT obtains the highest R@10. On HotpotQA, the first-stage graph baseline is already strong at R@5, and PathCondRAG mainly improves the deeper early pool, reaching the best R@10. The transfer datasets show a useful boundary: PathCondRAG gives the best PopQA recall, but Static QD with Qwen-Embedding is strongest on NQ retrieval, indicating that path conditioning is most valuable when bridge evidence must be recovered rather than when the answer is already well served by a single strong query.

A closer comparison with the two strongest structure-aware baselines is useful because both already expose graph or hierarchy information to retrieval. HippoRAG2 ties PathCondRAG at HotpotQA R@5, but PathCondRAG still improves HotpotQA R@10 from 0.8990 to 0.9260. On 2WikiMultiHopQA, where the evidence path is usually an explicit relation chain, PathCondRAG improves over HippoRAG2 by 7.02 points at R@5 and 10.54 points at R@10. RAPTOR is competitive on some single-hop and shallow cases, but it is consistently weaker on the compositional datasets where the

missing item must be reached through a bridge page. These differences support the mechanism claim: the gain comes less from making the index more structured in general and more from using path information to rewrite the next query.

The NQ and PopQA columns also help bound the claim. NQ rewards a strong lexical or dense query over multi-hop exploration, so Static QD with Qwen-Embedding remains slightly ahead. PopQA behaves differently: long-tail entity questions benefit from additional evidence exploration, and PathCondRAG obtains the best recall even though the task is not written as a multi-hop benchmark. We therefore treat the transfer results as a stress test rather than as the main proof of the method. The strongest evidence remains the paired pattern on MuSiQue, HotpotQA, and 2WikiMultiHopQA: when the question hides a bridge entity that is surfaced by the initial graph pass, conditioning decomposition on retrieved paths gives the reader a better chance to see the missing document within the top-5 or top-10 pool.

The downstream QA results follow the retrieval pattern with smaller margins, as expected because the reader can still fail even when the evidence is present. PathCondRAG gives the best EM and

Table 4: Controlled ablations of PathCondRAG. HippoRAG2 is included as the graph-backbone reference. The rows *w/o QD* and *w/o Path Hints* remove query decomposition and path conditioning, respectively, while keeping the reader and evidence budget fixed.

System	MuSiQue				HotpotQA				2Wiki			
	R@5	R@10	EM	F1	R@5	R@10	EM	F1	R@5	R@10	EM	F1
HippoRAG2	0.5780	0.6562	0.2260	0.3273	0.8415	0.8990	0.5140	0.6480	0.7405	0.7708	0.5110	0.5969
w/o QD	0.5588	0.6495	0.2500	0.3522	0.8250	0.8995	0.5130	0.6503	0.7455	0.7850	0.5030	0.5861
w/o Path Hints	0.5903	0.6857	0.2510	0.3564	0.8305	0.9120	0.5020	0.6376	0.7458	0.7920	0.5300	0.6038
PathCondRAG	0.6114	0.7178	0.2700	0.3714	0.8415	0.9260	0.5190	0.6536	0.8107	0.8762	0.5440	0.6356

F1 on MuSiQue and 2WikiMultiHopQA. On HotpotQA, it reaches the best F1 but not the best EM, indicating that path conditioning improves answer overlap more reliably than exact string matching in this setting. On NQ and PopQA, the strongest QA scores come from standard or iterative baselines rather than PathCondRAG. That pattern matches the method design: when no bridge entity is required, an extra decomposition step can improve recall without necessarily improving exact answer generation.

Two patterns are worth separating. First, PathCondRAG’s largest retrieval gains appear on datasets where a later evidence item is lexically distant from the original question but reachable through a bridge entity. The pattern is clearest on 2WikiMultiHopQA, where relation-chain structure makes the bridge explicit, and on MuSiQue, where composed single-hop facts must be assembled into a coherent answer path. Second, higher recall does not always translate into the best EM. NQ and PopQA show this mismatch: the reader can still prefer a competing answer string, or the added retrieved context can introduce distractors when the question is answerable from a direct passage. We interpret the main tables as evidence for a multi-hop retrieval mechanism rather than as a universal claim that path conditioning should dominate every factual QA setting.

5.3 Controlled Ablations of PC-QD

The main results establish aggregate gains, but they do not by themselves show whether the improvement comes from path conditioning or from the surrounding retrieval stack. We therefore use a controlled ablation suite in which every row uses the same corpus, graph retrieval backbone, retrieval budget, reader, and top-5 evidence budget. The key comparison is *PathCondRAG w/o Path Hints* versus *PathCondRAG*: both use query decomposition, but only the full model conditions decomposition on graph-derived path hints.

Table 4 shows that path conditioning, rather than decomposition alone, is the main source of the gain. Compared with *PathCondRAG w/o Path Hints*, the full model improves R@10 by 3.21 points on MuSiQue, 1.40 points on HotpotQA, and 8.42 points on 2Wiki. The corresponding F1 gains are 1.50, 1.60, and 3.18 points. *PathCondRAG w/o QD* provides a second control: the full model improves R@5 by 5.26, 1.65, and 6.52 points over this variant. Thus, graph retrieval alone and unconditioned decomposition each explain only part of the effect; the controlled comparison supports the path-conditioned mapping $f(q, \mathcal{P})$.

The ablation results also clarify why the method is not merely “more retrieval.” The *PathCondRAG w/o QD* variant keeps the graph retrieval backbone but removes the second query-generation channel, testing whether the initial graph neighborhood alone is enough.

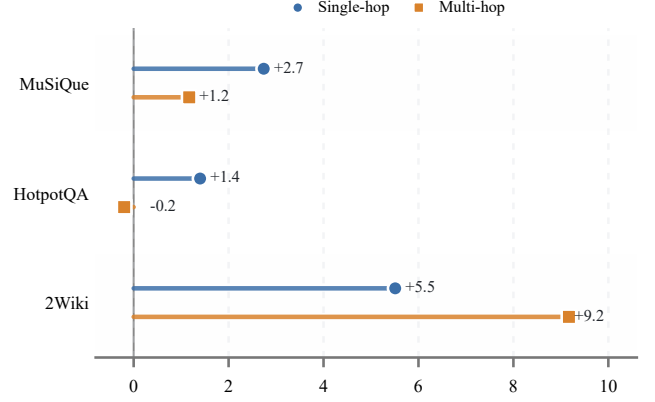


Figure 4: R@5 gain, in percentage points, of PathCondRAG over the w/o-path-hints ablation by hop group.

Table 5: PC-QD usage diagnostics from the ablation logs. Rates are computed over 1,000 questions per dataset.

Statistic	MuSiQue	HotpotQA	2Wiki
PC-QD used	88.2%	94.4%	93.5%
Avg. path hints	2.01	2.22	2.22
Entity replacement	56.3%	59.2%	85.7%
Fallback to dense retrieval	11.7%	5.6%	6.2%
Hint conflict rate	0.0%	0.0%	0.0%

The *PathCondRAG w/o Path Hints* variant restores decomposition but removes path conditioning, testing whether the decomposer can infer the bridge from the question surface form alone. The full model improves over both, indicating that the useful signal is the interaction between retrieved path structure and generated sub-questions. Thus, the gain is better explained by query specificity than by candidate-pool expansion alone.

5.4 Mechanism Diagnostics

Figure 4 summarizes the hop-stratified comparison most relevant to the mechanism claim: the R@5 gain over the w/o-path-hints ablation. PathCondRAG helps most on 2Wiki multi-hop questions, where bridge entities are explicit and relation chains are central to the task. The small negative HotpotQA multi-hop delta marks a boundary case and is consistent with the failure mode in which a plausible but non-gold bridge guides the decomposer away from the gold evidence.

Table 5 shows that PC-QD is active for most examples and receives about two path hints on average. The entity replacement rate is especially high on 2Wiki, matching the large hop-stratified gain in Figure 4. The fallback rate marks an important boundary: when the first graph pass extracts no usable facts, PathCondRAG falls back to dense passage retrieval rather than forcing a path-conditioned rewrite. The hint conflict rate measures parser-level conflicts among selected hints, not semantic correctness; semantically wrong but internally consistent hints are counted in the failure audit.

5.5 Qualitative Case Study

The aggregate tables show where PathCondRAG gains recall, but they do not show whether the extra evidence is actually the missing bridge needed by the reader. Table 6 traces one controlled HotpotQA example from decomposition to retrieved passages and reader output. The case is useful because both systems retrieve the seed film page at rank 1; the difference is whether the second-hop query is grounded enough to retrieve the director page.

We report the intermediate decomposition signal instead of only the final answer so that the failure can be inspected at the retrieval boundary. Static QD already asks a reasonable first-hop question, but its second-hop query preserves the role phrase “the director.” PathCondRAG receives a graph-derived cue linking *Panic 5 Bravo* to *Kuno Becker*, so the second-hop query names the bridge entity directly. This makes the example a compact version of the main mechanism: path conditioning does not change the reader or the evidence budget; it changes which document reaches the reader.

The case study is not intended as a substitute for the aggregate results; its role is to make the mechanism visible. Both systems retrieve the seed page *Panic 5 Bravo* at rank 1, so the first hop is already solved. The failure occurs in the second hop. Static QD keeps the phrase “the director” under-specified, and the reader never sees the *Kuno Becker* page that contains the birth year. PathCondRAG uses the retrieved path cue to ground that role as Kuno Becker, turns the follow-up into an entity-specific query, and moves the bridge document into rank 3. This isolates the mechanism: the first hop is unchanged, but grounding the second-hop query moves the missing bridge page into the reader context.

5.6 Failure Modes and Efficiency

Figure 5 summarizes a manual audit of 100 wrong MuSiQue predictions, with one primary failure category assigned to each example. We use the following diagnostic criteria. *Unreachable graph* denotes cases where the gold-supporting passage or a useful bridge entity is absent from the initial graph neighborhood. *Wrong path hint* denotes cases where the graph pass exposes a plausible but non-gold bridge, causing PC-QD to generate a misleading sub-question. *Reader failure* denotes cases where the required evidence appears in the final top-5 context but the reader still predicts an incorrect answer. *No-facts fallback* denotes cases where no usable facts are extracted and the system falls back to dense retrieval. *Too-deep path* denotes cases where the required evidence chain exceeds the configured retrieval or decomposition depth.

The largest error mass comes from graph unreachability, indicating that PathCondRAG cannot recover evidence that the initial

graph pass never surfaces. The second-largest category is wrong path hints, which is the main PathCondRAG-specific failure mode: when a plausible but incorrect bridge is selected, path conditioning can amplify that error by making the follow-up query more specific in the wrong direction. These errors define the method’s boundary condition. Path conditioning is useful when graph retrieval exposes a reliable bridge, but it is not a substitute for graph coverage or bridge validation.

PathCondRAG also introduces query-time overhead because it runs decomposition and sub-question retrieval in addition to the initial graph pass. All times are taken from the unified evaluation logs under the same experimental protocol. On 1,000 examples, PathCondRAG takes 6,754 seconds on MuSiQue, 6,148 seconds on HotpotQA, and 5,311 seconds on 2Wiki, while IRCot takes 9,430, 5,157, and 5,581 seconds, respectively. On the transfer datasets, PathCondRAG takes 6,405 seconds on NQ and 4,980 seconds on PopQA, compared with 7,024 and 5,585 seconds for IRCot. Because indexing pipelines and retrieval implementations differ across baselines, we treat these wall-clock numbers as coarse cost indicators rather than a strict speed ranking.

The failure and runtime analyses point to the same practical trade-off. PC-QD is most useful when the initial graph pass surfaces high-confidence path hints, but the extra decomposition call should not be treated as free. A natural extension is adaptive routing: apply PC-QD when the graph pass exposes reliable bridge evidence or when the question contains unresolved roles, and fall back to direct retrieval when the query is already entity-specific or the graph neighborhood is sparse. We leave this routing policy for future work because the present study isolates path-conditioned decomposition under a fixed protocol.

6 Limitations

PathCondRAG depends on the quality of the initial graph retrieval pass. If the graph does not contain the relevant entity, if entity extraction misses the bridge, or if the initial retrieval ranks the gold path too low, the decomposer may receive no useful path hint. In these cases, path-conditioned decomposition cannot recover evidence that is absent from the candidate graph neighborhood.

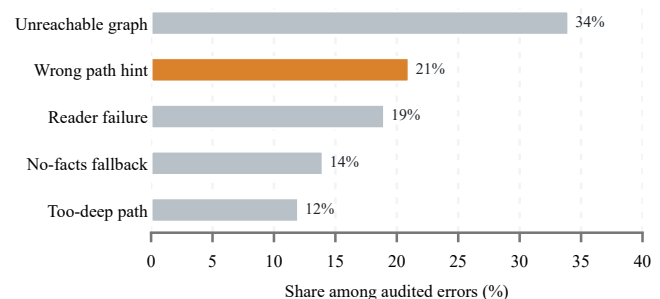


Figure 5: Failure distribution from a manual audit of 100 wrong MuSiQue predictions. Each example is assigned one primary failure category. The largest source is graph unreachability, while wrong path hints are the main PathCondRAG-specific failure mode.

Table 6: A controlled HotpotQA case study. PathCondRAG grounds the under-specified second hop and retrieves the missing bridge document.

Field	Static QD	PathCondRAG
Question	The director of <i>Panic 5 Bravo</i> was born in what year?	
Gold chain	<i>Panic 5 Bravo</i> → <i>Kuno Becker</i> → birth year 1978.	
Gold evidence	<i>Panic 5 Bravo</i> : "directed by Kuno Becker"; <i>Kuno Becker</i> : "born January 14, 1978".	
Decomposition signal	Original question only; no entity grounding from retrieved paths.	Path-conditioned cue from the seed passage linking <i>Panic 5 Bravo</i> to <i>Kuno Becker</i> .
Generated sub-questions	1. Who is the director of <i>Panic 5 Bravo</i> ? 2. In what year was the director of <i>Panic 5 Bravo</i> born?	1. Who directed the film <i>Panic 5 Bravo</i> ? 2. In what year was <i>Kuno Becker</i> , the director of <i>Panic 5 Bravo</i> , born?
Grounded entities	None; the second sub-question keeps the ambiguous phrase "the director".	<i>Panic 5 Bravo</i> , <i>Kuno Becker</i> ; <i>Kuno Becker</i> , <i>Panic 5 Bravo</i> .
Top-5 retrieved	1. <i>Panic 5 Bravo</i> (0.9982) 2. <i>Sex and Breakfast</i> (0.2212) 3. <i>Nico B.</i> (0.1400) 4. <i>Patriots Day (film)</i> (0.1380) 5. <i>Bruce Brown</i> (0.0970)	1. <i>Panic 5 Bravo</i> (0.9982) 2. <i>Patriots Day (film)</i> (0.2686) 3. <i>Kuno Becker</i> (0.2563) 4. <i>Sex and Breakfast</i> (0.1854) 5. <i>Nico B.</i> (0.1719)
Evidence coverage	1/2 gold documents in top-5; bridge document <i>Kuno Becker</i> is absent.	2/2 gold documents in top-5; bridge document <i>Kuno Becker</i> appears at rank 3.
Reader output	Fails to answer.	1978.

Note: Bold retrieved titles are gold evidence documents. Scores are rerun retrieval scores for HotpotQA sample index 881 in the 1,000-question evaluation file.

The method also depends on the decomposer’s ability to use structured hints without over-trusting noisy ones. The current experiments use one main LLM backbone, Qwen3-30B-A3B. Additional experiments with smaller open models and stronger API models would clarify how robust the conditioning effect is across decomposer capacity.

Path-conditioned decomposition can also improve retrieval recall without improving answer exact match. The NQ and PopQA results show this boundary clearly: PathCondRAG can retrieve competitive or even stronger early evidence, but the reader may still prefer a different answer string or be distracted by additional context. These results motivate answer-aware context selection or compression alongside path-conditioned retrieval.

The main comparison is limited to methods with runnable or verifiable implementations under our unified evaluation protocol. The constraint keeps the reported tables auditable, but additional path-pruning and graph-RAG systems should still be evaluated under the same indexing, retrieval-budget, and reader protocol before drawing broader conclusions about relative ranking among graph-RAG systems. Auxiliary rerankers such as MPCE are treated as analysis components rather than part of the core method.

7 Conclusion

This paper studied the interface between graph-enhanced retrieval and query decomposition in multi-hop RAG. PathCondRAG uses graph-derived path hints to condition the decomposer before the next retrieval query is generated, changing decomposition from $f(q)$ to $f(q, \mathcal{P})$. This makes bridge entities exposed by the initial graph pass available to query generation rather than only to passage ranking or reader context.

Across three primary multi-hop benchmarks, PathCondRAG improves early evidence recall and downstream QA over graph retrieval and static decomposition variants. Transfer results on NQ and PopQA give a narrower picture: path conditioning can improve recall, but single-hop factual QA may still favor a strong direct retriever, and recall gains do not always translate into exact-match gains. Controlled ablations show that the main improvement comes from conditioning decomposition on path hints rather than from enabling decomposition alone, while the failure analysis shows the

expected boundary: the method depends on the initial graph pass surfacing a useful bridge.

Future work will extend the comparison to more graph-RAG and path-pruning systems under the same verified protocol, evaluate additional LLM backbones, and study multi-round path-hint extraction and decomposition for questions requiring more than two evidence hops.

GenAI Usage Disclosure

The authors acknowledge the limited use of generative AI tools during manuscript preparation. ChatGPT was used only for minor language editing, such as spelling, grammar, punctuation correction, and improving sentence-level clarity, in a manner similar to standard writing aids such as spell-checkers or grammar tools.

Claude 4.6 Sonnet was used only as an auxiliary tool for preparing diagram and visualization drafts based on author-provided descriptions, layouts, data, and specifications. The final figures were selected, checked, revised, and approved by the authors.

All experimental design, implementation, data analysis, and interpretation of results were conducted entirely by the authors without generative AI assistance. The core research ideas, framework architecture, mathematical formulations, and technical contributions represent original work by the authors. This use of generative AI tools complies with ACM policy and does not compromise the originality, validity, or integrity of the research.

References

- [1] Akari Asai, Kazuma Hashimoto, Hannaneh Hajishirzi, Richard Socher, and Caiming Xiong. 2020. Learning to Retrieve Reasoning Paths over Wikipedia Graph for Question Answering. In *Proceedings of the 8th International Conference on Learning Representations*.
- [2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *Proceedings of the International Conference on Learning Representations*. <https://openreview.net/forum?id=hSyW5go0v8>
- [3] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. 2022. Improving Language Models by Retrieving from Trillions of Tokens. In *Proceedings of the 39th International Conference on Machine Learning*. 2206–2240.
- [4] Boyu Chen, Zirui Guo, Zidan Yang, Yuluo Chen, Junze Chen, Zhenghao Liu, Chuan Shi, and Cheng Yang. 2026. PathRAG: Pruning Graph-based Retrieval

- Augmented Generation with Relational Paths. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 30183–30191. doi:10.1609/aaai.v40i36.40268
- [5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130 [cs.CL] <https://arxiv.org/abs/2404.16130>
 - [6] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise Zero-Shot Dense Retrieval without Relevance Labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1762–1777. doi:10.18653/v1/2023.acl-long.99
 - [7] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2025. LightRAG: Simple and Fast Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, Suzhou, China, 10746–10761. doi:10.18653/v1/2025.findings-emnlp.568
 - [8] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 37. <https://proceedings.neurips.cc/paper/2024/hash/6ddc001d07ca4f319af9a3024f6dbd1-Abstract.html>
 - [9] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 21497–21515. <https://proceedings.mlr.press/v267/gutierrez25a.html>
 - [10] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-Augmented Language Model Pre-Training. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 3929–3938. <https://proceedings.mlr.press/v119/guu20a.html>
 - [11] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *Proceedings of the 28th International Conference on Computational Linguistics*. International Committee on Computational Linguistics, Barcelona, Spain (Online), 6609–6625. doi:10.18653/v1/2020.coling-main.580
 - [12] Gautier Izacard and Edouard Grave. 2021. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 874–880. doi:10.18653/v1/2021.eacl-main.74
 - [13] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2023. Atlas: Few-shot Learning with Retrieval Augmented Language Models. *Journal of Machine Learning Research* 24, 251 (2023), 1–43.
 - [14] Soyeong Jeong, Jinheon Bae, Sukmin Cho, Sung Ju Hwang, and Jong Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, Mexico City, Mexico, 7036–7050. doi:10.18653/v1/2024.naacl-long.389
 - [15] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active Retrieval Augmented Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 7969–7992. doi:10.18653/v1/2023.emnlp-main.495
 - [16] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1601–1611. doi:10.18653/v1/P17-1147
 - [17] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Online, 6769–6781. doi:10.18653/v1/2020.emnlp-main.550
 - [18] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. Association for Computing Machinery, Virtual Event, China, 39–48. doi:10.1145/3397271.3401075
 - [19] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2023. Decomposed Prompting: A Modular Approach for Solving Complex Tasks. In *Proceedings of the 11th International Conference on Learning Representations*.
 - [20] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466. doi:10.1162/tac1_a_00276
 - [21] Kwun Hang Lau, Fangyuan Zhang, Boyu Ruan, Yingli Zhou, Qintian Guo, Ruiyuan Zhang, and Xiaofang Zhou. 2026. Breaking the Static Graph: Context-Aware Traversal for Robust Retrieval-Augmented Generation. arXiv:2602.01965 [cs.CL] <https://arxiv.org/abs/2602.01965>
 - [22] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
 - [23] Hao Liu, Zhengren Wang, Xi Chen, Zhiyu Li, Feiyu Xiong, Qinhan Yu, and Wentao Zhang. 2025. HopRAG: Multi-Hop Reasoning for Logic-Aware Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, Vienna, Austria, 1897–1913. doi:10.18653/v1/2025.findings-acl.97
 - [24] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query Rewriting in Retrieval-Augmented Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 5303–5315. doi:10.18653/v1/2023.emnlp-main.322
 - [25] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Toronto, Canada, 9802–9822. doi:10.18653/v1/2023.acl-long.546
 - [26] Sewon Min, Victor Zhong, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2019. Multi-hop Reading Comprehension through Question Decomposition and Rescoring. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 6097–6109. doi:10.18653/v1/P19-1613
 - [27] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. 2021. KILT: a Benchmark for Knowledge Intensive Language Tasks. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2523–2544. doi:10.18653/v1/2021.naacl-main.200
 - [28] Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, and Alexander Miller. 2019. Language Models as Knowledge Bases?. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2463–2473. doi:10.18653/v1/D19-1250
 - [29] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. 2023. Measuring and Narrowing the Compositionality Gap in Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, Singapore, 5687–5711. doi:10.18653/v1/2023.findings-emnlp.378
 - [30] Peng Qi, Xiaowen Lin, Leo Mehr, Zijian Wang, and Christopher D. Manning. 2019. Answering Complex Open-domain Questions Through Iterative Query Generation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2590–2602. doi:10.18653/v1/D19-1261
 - [31] Adam Roberts, Colin Raffel, and Noam Shazeer. 2020. How Much Knowledge Can You Pack Into the Parameters of a Language Model?. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. 5418–5426. doi:10.18653/v1/2020.emnlp-main.437
 - [32] Stephen E. Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389. doi:10.1561/15000000019
 - [33] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022. ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Seattle, United States, 3715–3734. doi:10.18653/v1/2022.naacl-main.272
 - [34] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. In *Proceedings of the International Conference on Learning Representations*. <https://openreview.net/forum?id=8a2acd174940dbca361a6398a4f9df91> Published as a conference paper at ICLR 2024.
 - [35] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*.
 - [36] Haitian Sun, Tania Bedrax-Weiss, and William W. Cohen. 2019. PullNet: Open Domain Question Answering with Iterative Retrieval on Knowledge Bases and Text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, Hong

- Kong, China, 2380–2390. doi:10.18653/v1/D19-1242
- [37] Haitian Sun, Bhuwan Dhingra, Manzil Zaheer, Kathryn Mazaitis, Ruslan Salakhutdinov, and William W. Cohen. 2018. Open Domain Question Answering Using Early Fusion of Knowledge Bases and Text. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 4231–4242.
- [38] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*.
- [39] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554. doi:10.1162/tacl_a_00475
- [40] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Toronto, Canada, 10014–10037. doi:10.18653/v1/2023.acl-long.557
- [41] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*.
- [42] Johannes Welbl, Pontus Stenetorp, and Sebastian Riedel. 2018. Constructing Datasets for Multi-hop Reading Comprehension Across Documents. *Transactions of the Association for Computational Linguistics* 6 (2018), 287–302. doi:10.1162/tacl_a_00021
- [43] Tomer Wolfson, Mor Geva, Ankit Gupta, Matt Gardner, Yoav Goldberg, Daniel Deutch, and Jonathan Berant. 2020. BREAK It Down: A Question Understanding Benchmark. *Transactions of the Association for Computational Linguistics* 8 (2020), 183–198. doi:10.1162/tacl_a_00309
- [44] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 2369–2380. doi:10.18653/v1/D18-1259
- [45] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the International Conference on Learning Representations*.
- [46] Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. 2021. QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 535–546. doi:10.18653/v1/2021.naacl-main.45
- [47] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. 2023. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. In *Proceedings of the 11th International Conference on Learning Representations*.